



CLOUD & DEVOPS

DEVOPS PROJECT:

Deploying an Open-Source Large Language Model (LLM)

Présenté par : Fatima DIEYE

Professeur : Ibrahima MBENGUE

DIC 2 GIT - Année scolaire 2025/2026

DE L'IDÉE AU CLUSTER

Tout commence avec une idée simple : faire tourner un modèle de langage chez moi, sur ma propre machine, sans API externe, sans abonnement, sans dépendre de personne. Juste moi, Docker, et quelques gigaoctets de poids de modèle qui se mettent à tourner en local.

Sauf que **"faire tourner" et "faire tourner correctement" sont deux choses complètement différentes, et c'est là que le vrai projet commence.** Ça veut dire une infrastructure qui ne s'écroule pas au premier redémarrage, des services qui savent vérifier leur propre santé, des ressources allouées avec intelligence plutôt qu'au hasard. Ça veut dire passer de "ça marche sur ma machine" à "ça marche, point final, peu importe ce qui se passe."

L'aventure se déroule en deux temps. D'abord Docker Compose, pour comprendre l'architecture : comment Ollama et Open WebUI se parlent, ce dont chacun a besoin pour exister. Puis Kubernetes, où tout change de philosophie, on ne lance plus une application, on déclare l'état dans lequel elle doit être, et on laisse le cluster se débrouiller pour l'y maintenir envers et contre tout.

Ce qui suit, c'est le déroulé complet de cette aventure : les étapes techniques, les captures d'écran, et après chacune, ce que j'en ai vraiment retenu, les blocages, les surprises, les petites victoires devant un terminal à 23h <3

Tâche 1 : lancer l'application avec Docker Compose.

Une fois docker compose up -d lancé, j'ai créé le compte administrateur avec mon adresse dieyef@ept.edu.sn, devenue admin automatiquement en tant que premier compte créé sur l'instance. Tout est stocké localement dans le volume open_webui_data, donc persistant même après redémarrage.

Téléchargement des modèles

Les trois modèles requis ont été téléchargés dans le conteneur Ollama avec les commandes suivantes :

```
bash
docker compose exec ollama ollama pull llama3.2:3b
docker compose exec ollama ollama pull mistral:7b
docker compose exec ollama ollama pull deepseek-r1:8b
```

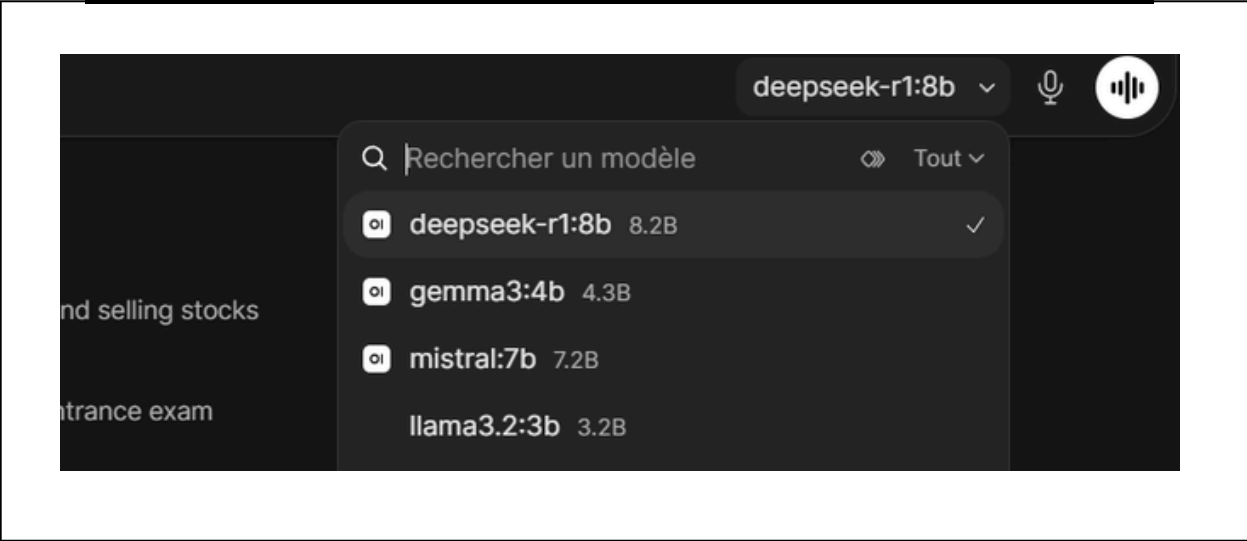
Les trois modèles apparaissent ensuite dans le sélecteur d'Open WebUI, et chacun répond correctement à un prompt de test :

Modèle	Taille approx.
llama3.2:3b	2,0 Go
mistral:7b	4,1 Go
deepseek-r1:8b	5,2 Go

PARTIE 01

DÉPLOIEMENT LOCAL AVEC DOCKER COMPOSE

Modèles Téléchargés



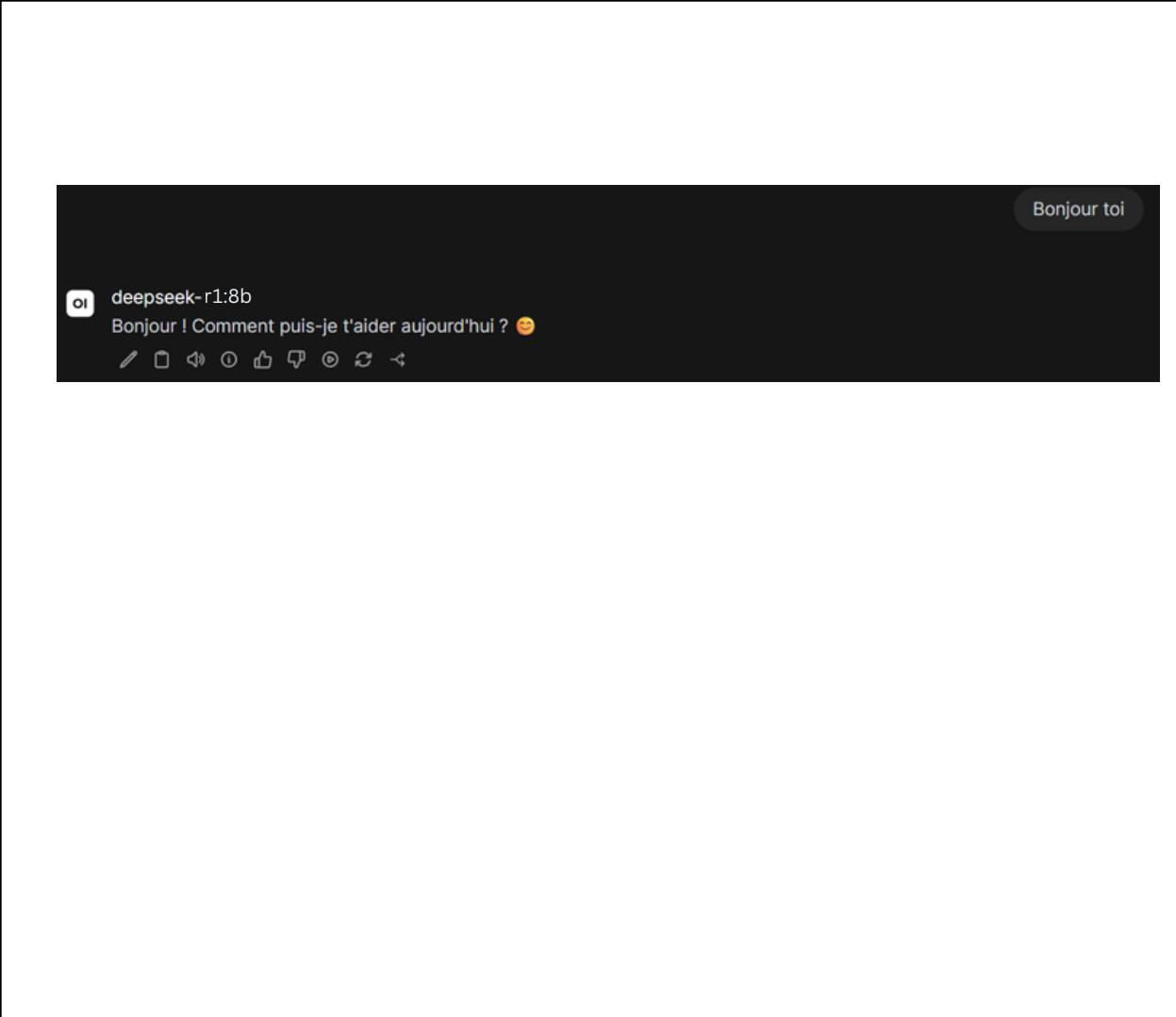
mistral:7b

**Deepseek-
r1:8b**

Llama 3.1:3b

**Deepseek-
r1:8b**

Llama 3.1:3b



PARTIE 01

DÉPLOIEMENT LOCAL AVEC DOCKER COMPOSE	
Modèles Téléchargés	
mistral:7b	
Deepseek-r1:8b	
Llama 3.1:3b	
PARTIE 01	

Passer du conteneur au cluster

Docker Compose, c'était lancer des conteneurs et les laisser tourner. Kubernetes change complètement de logique : on déclare l'état voulu, et le cluster se charge de le maintenir tout seul — redémarrage automatique si un pod crash, vérifications de santé continues, gestion déclarative du stockage et du réseau.

J'ai commencé par arrêter les conteneurs du Compose (`docker compose down`) pour libérer de la RAM, puis installé `kubectl` et `minikube`. Vu mes 8 Go de RAM, j'ai démarré le cluster avec des ressources volontairement réduites :

```
bash
minikube start --driver=docker --memory=3000 --cpus=2
```

Minikube crée le cluster via Docker Desktop, gardé ouvert en arrière-plan.

Ensuite, j'ai écrit tous les manifests YAML nécessaires : un Namespace dédié (`llm-app`), un ConfigMap pour la configuration d'Open WebUI, deux PersistentVolumeClaims (un par service, pour garder les modèles et les données même après redémarrage), deux Deployments avec `resource requests/limits` et les trois probes (`startup`, `readiness`, `liveness`), deux Services, et un Ingress pour exposer l'interface.

Installation du cluster local

```
PS C:\WINDOWS\system32> minikube start --driver=docker --memory=3000 --cpus=2
minikube v1.39.0 on Microsoft Windows 11 Home 25H2
g the docker driver based on user configuration
g Docker Desktop driver with root privileges
ting "minikube" primary control-plane node in "minikube" cluster
ing base image v0.0.51 ...
loading Kubernetes v1.37.0 preload ...
preloaded-images-k8s-v18-v1...: 347.58 MiB / 347.58 MiB 100.00% 8.07 Mi
gcr.io/k8s-minikube/kicbase: 507.61 MiB / 507.61 MiB 100.00% 6.97 MiB p
ting docker container (CPUs=2, Memory=3000MB) ...
uting "docker container inspect minikube --format={{.State.Status}}" took an unusually long time: 2.0988632s
starting the docker service may improve performance.
paring Kubernetes v1.37.0 on containerd 2.3.4 ...
figuring CNI (Container Networking Interface) ...
fyding Kubernetes components...
ing image gcr.io/k8s-minikube/storage-provisioner:v5
```

```
PS C:\WINDOWS\system32> kubectl get nodes
NAME          STATUS    ROLES          AGE   VERSION
minikube      Ready     control-plane  28m   v1.37.0
PS C:\WINDOWS\system32>
```

PARTIE 02

DÉPLOIEMENT KUBERNETES

Application
de tous les
manifests

```
PS C:\users\fatima\Documents\projet_devops> cd k8s
PS C:\users\fatima\Documents\projet_devops\k8s> kubectl apply -f .
namespace/llm-app created
configmap/open-webui-config created
persistentvolumeclaim/ollama-pvc created
persistentvolumeclaim/open-webui-pvc created
deployment.apps/ollama created
deployment.apps/open-webui created
service/ollama created
service/open-webui created
ingress.networking.k8s.io/open-webui-ingress created
PS C:\users\fatima\Documents\projet_devops\k8s> █
```

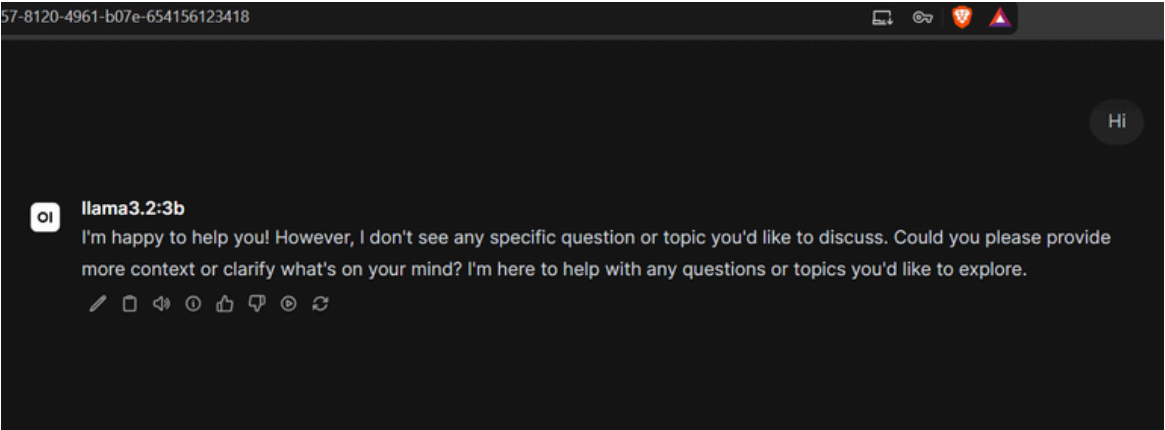
running
pods

```
PS C:\users\fatima\Documents\projet_devops\k8s > kubectl get pods -n projet-devops
NAME                                READY   STATUS    RESTARTS   AGE
ollama-7cc7f75df7-k5c45             1/1     Running   0           10m
open-webui-897466794-smbh6          1/1     Running   5 (2m7s ago)  8m42s
```

running
services

```
PS C:\users\fatima\Documents\projet_devops\k8s > kubectl get services -n projet-devops
NAME                TYPE        CLUSTER-IP    EXTERNAL-IP  PORT(S)    AGE
ollama-service      ClusterIP   10.108.81.202 <none>       80/TCP     3m7s
open-webui-service  ClusterIP   10.102.165.73 <none>       80/TCP     3m7s
```

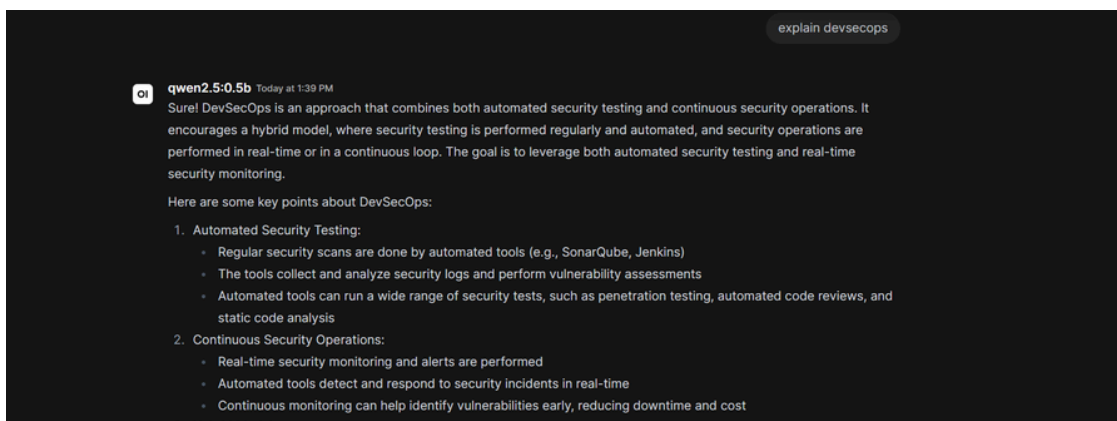
Llama3:2:1b
prompted



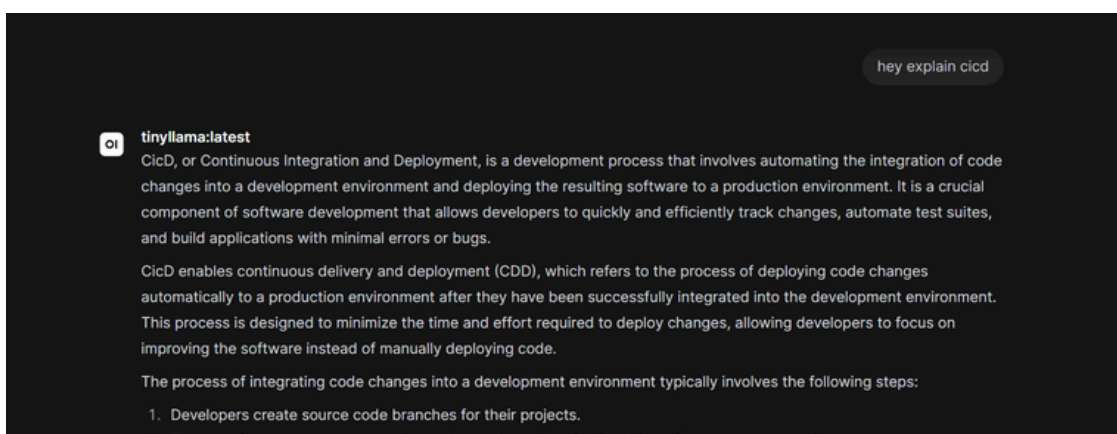
PARTIE 01

DÉPLOIEMENT KUBERNETES

qwen2.5:0.5b
prompted



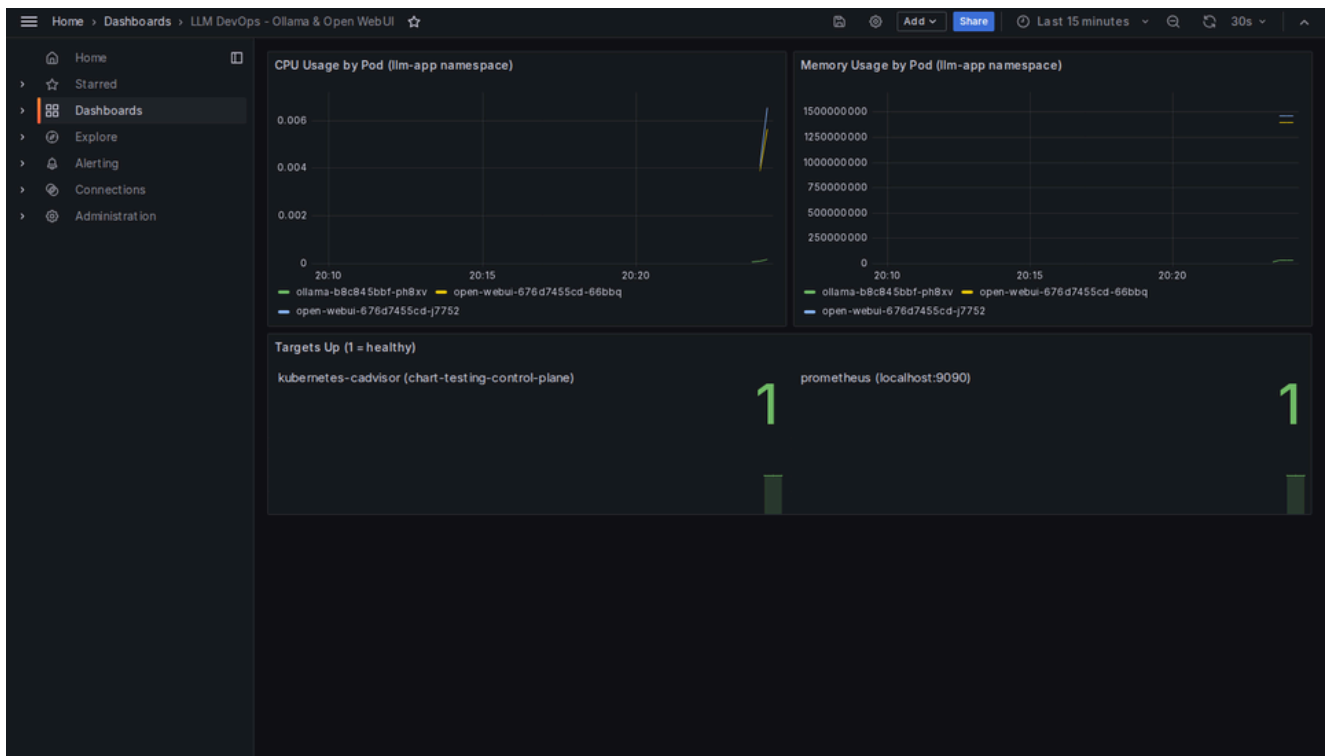
tinyllama:
latest
prompted



Le déploiement Kubernetes utilise trois modèles légers : **llama3.2:3b** (environ 2,0 Go), **qwen2.5:0.5b** (environ 0,4 Go) et **tinyllama:latest** (environ 0,6 Go). Ollama charge le modèle entier en RAM, en plus de la mémoire utilisée par Open WebUI et les composants système de Kubernetes, et le nœud minikube dispose d'une mémoire limitée. Des modèles plus lourds comme mistral:7b (environ 4,1 Go) ou deepseek-r1:8b (environ 5,2 Go) dépasseraient la mémoire disponible pour le conteneur Ollama. Les trois modèles retenus couvrent des familles et des tailles différentes, ce qui suffit à vérifier qu'Ollama sert plusieurs modèles, qu'ils apparaissent tous dans Open WebUI et que chacun répond aux prompts, tout en respectant la limite mémoire définie dans le Deployment.

PARTIE 02

Dashboard Graphana



Prometheus Monitoring

```

✓ Verify Prometheus targets and query real metrics

1 ▶ Run kubectl -n monitoring port-forward svc/prometheus 9090:9090 &
22 Forwarding from 127.0.0.1:9090 -> 9090
23 Forwarding from [::1]:9090 -> 9090
24 --- Waiting for at least 2 healthy targets ---
25 Handling connection for 9090
26 Healthy targets so far: 2
27 === Prometheus targets (raw JSON) ===
28 Handling connection for 9090
29 {
30   "status": "success",
31   "data": {
32     "activeTargets": [
33       {
34         "discoveredLabels": {
35           "__address__": "172.18.0.2:10250",
36           "__meta_kubernetes_node_address_hostname": "chart-testing-control-plane",
37           "__meta_kubernetes_node_address_internalip": "172.18.0.2",
38           "__meta_kubernetes_node_annotation_node_alpha_kubernetes_io_ttl": "0",
39           "__meta_kubernetes_node_annotation_volumes_kubernetes_io_controller_managed_attach_detach": "true",
40           "__meta_kubernetes_node_annotationpresent_node_alpha_kubernetes_io_ttl": "true",
41           "__meta_kubernetes_node_annotationpresent_volumes_kubernetes_io_controller_managed_attach_detach": "true",
42           "__meta_kubernetes_node_label_beta_kubernetes_io_arch": "amd64",
43           "__meta_kubernetes_node_label_beta_kubernetes_io_os": "linux",

```

CONCLUSION

Ce projet a permis de migrer un déploiement Docker Compose vers Kubernetes, en recréant l'ensemble de l'architecture (Namespace, Deployments, Services, ConfigMaps, PersistentVolumeClaims et Ingress) tout en respectant les bonnes pratiques attendues : limites de ressources, sondes de santé à trois niveaux, et stockage persistant garantissant qu'aucune donnée n'est perdue lors d'un redémarrage.

La contrainte la plus significative a été la mémoire disponible sur la machine de développement, qui a orienté plusieurs choix techniques : des délais de sonde plus tolérants, un seul replica par application pour éviter toute concurrence sur les volumes, et une sélection de modèles adaptée à cette limite. Ces ajustements illustrent une réalité du DevOps : concevoir une infrastructure fiable, ce n'est pas seulement écrire des manifestes corrects, c'est aussi les adapter aux contraintes réelles de l'environnement où ils s'exécutent.

Le résultat final répond à l'ensemble des critères de validation : les deux applications tournent de façon stable, communiquent correctement via le réseau interne de Kubernetes, et les modèles déployés répondent aux requêtes des utilisateurs à travers Open WebUI.